

PRUEBA TÉCNICA · CIBERSEGURIDAD

Decisiones de Arquitectura (ADR)

Registro de decisiones clave

PREPARADO POR	Lady Marcela Romero Rivero
ROL	Ingeniero de Ciberseguridad
CLIENTE	Simon Movilidad
ESCENARIO	FleetSec S.A.S. — Telemetría de flotas
FECHA	26 de agosto de 2026
VERSIÓN	1.0

CONFIDENCIAL — Exclusivo para Simon Movilidad

Tabla de contenido

ADR-0001 — Stack: Node.js 20 + Express, aplicación vulnerable mínima propia

Contexto

Decisión

Consecuencias

ADR-0002 — Puertas de calidad y umbrales de DevSecOps

Contexto

Decisión

Consecuencias

ADR-0003 — Línea base de seguridad de Terraform y excepciones de Checkov documentadas

Contexto

Decisión

Consecuencias

ADR-0004 — Enfoque de ingeniería de detección y respuesta a incidentes

Contexto

Decisión

Consecuencias

ADR-0005 — Gestión de las «trampas» embebidas en los requisitos de la prueba

Contexto

Decisión (trampa → resolución)

Consecuencias

Registro de las decisiones técnicas clave del programa de seguridad FleetSec.

ADR-0001 — Stack: Node.js 20 + Express, aplicación vulnerable mínima propia

• **Estado:** Aceptado • **Fecha:** 2026-08-23 • **Responsable:** Ingeniería de Seguridad

Contexto

La prueba permite elegir libremente el stack y utilizar, bien una aplicación vulnerable conocida (Juice Shop, DVWA), bien una construcción propia mínima para el VAPT. Debemos demostrar las 10 clases CWE específicas y remediar ≥ 8 **en nuestro propio código** con pruebas emparejadas maliciosas/legítimas.

Decisión

Construir una aplicación mínima en Express (Node 20) con una superficie de laboratorio `/vuln` (los 10 CWE) y un gemelo remediado `/secure`. Usar dependencias en JavaScript puro (`sql.js` WASM SQLite, sin compilación nativa) para mantener la imagen Docker ligera y el pipeline rápido.

Consecuencias

- (+) Controlamos exactamente los 10 CWE; cada PoC y cada corrección es reproducible y está cubierta por pruebas unitarias (20/20).
- (+) La remediación reside en nuestro código (la rúbrica lo premia frente a parchear aplicaciones de terceros).
- (+) `docker compose up` es un único comando rápido (Bonus +5%).
- (–) Un laboratorio construido a mano debe documentarse de forma transparente (p. ej., el parser XXE es deliberadamente ingenuo); se deja constancia en el README y en el hallazgo.
- (–) No es un objetivo «famoso»; se mitiga mapeando cada hallazgo a OWASP/CWE y ofreciendo Juice Shop como banco de comparación opcional.

ADR-0002 — Puertas de calidad y umbrales de DevSecOps

• **Estado:** Aceptado • **Fecha:** 2026-08-23 • **Responsable:** Ingeniería de Seguridad

Contexto

El pipeline debe imponer puertas estrictas (SAST, SCA, DAST, contenedor, IaC, secretos), pero el repositorio contiene de forma intencionada un laboratorio vulnerable. Una puerta ingenua de «0 hallazgos» fallaría de forma permanente o nos obligaría a eliminar el material didáctico.

Decisión

SAST de dos niveles: una ejecución **informativa** de Semgrep sobre todo el repositorio (muestra las reglas personalizadas activándose, no bloqueante) más una puerta **bloqueante** solo sobre el código de producción (`app/src` , excluyendo `vulnerable.js` y las pruebas), con supresiones `nosemgrep` en línea que incluyen `razón · fecha · responsable` para el secreto de laboratorio V-10. Umbrales: SCA bloquea HIGH/CRITICAL ($\approx$ CVSS ≥ 8 directo / ≥ 9 indirecto), el contenedor bloquea CRITICAL, DAST bloquea HIGH/CRITICAL + MEDIUM abre una incidencia `security/medium` , Checkov falla de forma estricta (los controles integrados CKV_AWS_24/25 imponen la regla del puerto de administración). Los jobs independientes se ejecutan en paralelo con cachés de npm/Trivy para mantenerse ≤ 15 min.

Consecuencias

- (+) La puerta refleja el riesgo real de producción; las supresiones son auditables (replica la disciplina SAST exigida).
- (+) Las reglas personalizadas siguen ejercitándose de forma demostrable.
- (+) El fan-out en paralelo + el cacheo cumplen el SLA de ≤ 15 min.
- (–) La lista de exclusiones debe revisarse para que nunca oculte un hallazgo real de producción — bajo responsabilidad de SecOps en CODEOWNERS.

ADR-0003 — Línea base de seguridad de Terraform y excepciones de Checkov documentadas

- **Estado:** Aceptado · **Fecha:** 2026-08-23 · **Responsable:** Ingeniería de Seguridad / Plataforma Cloud

Contexto

El Entregable 03 pide un módulo `modules/security-baseline` que modele una base segura de AWS, con `terraform validate` limpio y un análisis de Checkov. Algunos ajustes solicitados son en realidad **trampas** (véase ADR-0005) y algunos controles de Checkov quedan fuera del alcance de una línea base de red/datos (p. ej., los recursos ALB/ECS residen en el stack de la aplicación).

Decisión

Escribir un módulo autocontenido (VPC de 3 capas, CMK de KMS, S3 + Object Lock, RDS Multi-AZ, Secrets Manager, CloudTrail, Config, GuardDuty, Security Hub, WAFv2) cuyos SG son consumidos por el stack de la aplicación mediante outputs. Llevar Checkov a **0 fallidos** corrigiendo cada brecha real de bajo costo (bloqueo del SG por defecto, logging de accesos, autenticación IAM/PI/monitoreo mejorado en RDS, ciclo de vida, SNS de CloudTrail, logging de WAF) y añadiendo **12 skips documentados en línea** (`checkov:skip=ID:reason`) para elementos intencionales/fuera de alcance (p. ej., la redirección del puerto 80 del ALB por requerimiento, la replicación entre regiones como riesgo aceptado, la política de la clave raíz de KMS). Añadir una barrera de protección personalizada (CKV2_FLEETSEC_1 en YAML + Python con pruebas unitarias).

Consecuencias

- (+) El resultado de `checkov` es de calidad de auditoría: 254 aprobados / 0 fallidos / 12 skips documentados.
- (+) La trampa del puerto de administración se impone mediante controles integrados (CKV_AWS_24/25) que APRUEBAN nuestros SG y RECHAZAN un SG defectuoso.
- (–) `terraform validate / plan` completos deben ejecutarse localmente (sin credenciales cloud en CI); la sintaxis HCL se validó offline para los 12 archivos.
- (–) Los diagramas se entregan como imágenes renderizadas (PNG, generadas con la librería `diagrams` + Graphviz), no como fuente Mermaid.

ADR-0004 — Enfoque de ingeniería de detección y respuesta a incidentes

- **Estado:** Aceptado · **Fecha:** 2026-08-23 · **Responsable:** Ingeniería de Seguridad

Contexto

El Entregable 04 proporciona una cronología de la brecha totalmente embebida. Debemos producir IOCs, detecciones, un playbook de IR, mapeo MITRE, RCA y comunicaciones ejecutivas.

Decisión

Priorizar detecciones situadas en la parte alta de la **Pirámide del Dolor** (comportamientos/TTP por encima de IPs): 4 reglas Sigma (administración IAM fuera de horario, DeleteTrail antiforenses CRITICAL, SQLi en los logs de la aplicación, GetObject masivo de S3). Cargar los IOCs en un **Threat Intel Set** de GuardDuty mediante Terraform (preferido) con un método alternativo por AWS CLI. La IR sigue NIST 800-61 (preservar→contener→erradicar→recuperar→lecciones) con AWS CLI reversible y exacto, MITRE ATT&CK v14 (8 técnicas), RCA con 5 Porqués + queso suizo, un informe para el CEO de ≤1 página y la ruta de notificación a la SIC de la Ley 1581 (15 días hábiles).

Consecuencias

- (+) Reglas validadas con `pysigma` (parseo limpio, 4/4). Un *intento* de DeleteTrail genera alerta incluso cuando la SCP lo bloquea.
- (+) El Threat Intel Set está bajo control de versiones y es reproducible.
- (–) La sintaxis de agregación de Sigma (GetObject masivo) depende del backend — documentado; se traduce con `sigma convert`.
- (–) Los IOCs basados en IP (185.220.101.22) caducan rápido — reevaluar cada 90 días; las reglas de comportamiento cargan con el peso.

ADR-0005 — Gestión de las «trampas» embebidas en los requisitos de la prueba

• **Estado:** Aceptado • **Fecha:** 2026-08-23 • **Responsable:** Ingeniería de Seguridad

Contexto

Varios requisitos están redactados deliberadamente para tentar a una implementación insegura o no conforme. Una ingeniera sénior debe detectarlos y corregir en lugar de cumplir a ciegas. Este ADR registra cada trampa y la decisión profesional adoptada. Análisis completo en `README.md` §"Análisis crítico y trampas".

Decisión (trampa → resolución)

1. **KMS "key policy sin Principal AWS *"** — un comodín `Principal: "*"` en una política de clave es peligroso. Usamos la sentencia de administración con el root de la cuenta + principales de servicio acotados (nunca `Principal: "*"`). Véase `kms.tf`.
2. **SG "0.0.0.0/0 excepto 80/443 en ALB"** — solo el SG del ALB expone 0.0.0.0/0, únicamente en el 80 (redirección) y el 443; los puertos de administración nunca. Impuesto por CKV_AWS_24/25.
3. **RDS `ssl=1` / `log_connections=1`** — implementado mediante un parameter group (`rds.force_ssl=1`, `log_connections=1`), más autenticación IAM y sin endpoint público.
4. **V-10 "migrar a variable de entorno o gestor de secretos — nunca mover a otro archivo del repo"** — mover un secreto a otro archivo del repositorio es la trampa; lo trasladamos a variable de entorno/Secrets Manager. Véase `secure.js` / `rds.tf`.
5. **DAST "cobertura ≥80% del OpenAPI"** — publicamos una especificación OpenAPI de la superficie *segura* para que ZAP tenga un contrato real contra el cual autenticarse.
6. **Break-glass** — debe auditarse: 2 revisores mediante un Environment protegido + una incidencia CRITICAL abierta automáticamente en cada override (`break-glass.yml`).
7. **Informe de IA** — debe **admitir** al menos una alucinación de la IA; ocultarla puntúa cero. Documentado con honestidad en `docs/ai-report.md`.
8. **Cuenta de servicio `svc-monitoring` con acceso por consola** — las cuentas de servicio no deben tener acceso por consola; el plan de IR lo revoca y el backlog P1 añade una SCP.

Consecuencias

- (+) Demuestra criterio de seguridad por encima del cumplimiento literal — el núcleo de la evaluación.
- (+) Cada corrección es trazable hasta el código/la evidencia.
- (–) Un revisor que espere cumplimiento literal puede necesitar la justificación — de ahí este ADR y la sección del README.